.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica For The Win

13 de octubre de 2026

Altieri Franco — 105400
Santino Federico Bouza — 113991
Claudia Lis Ramos Sarzuri — 104596

1. Introducción

1.1. Contexto del problema

El presente trabajo continúa la situación planteada en el trabajo práctico anterior. Sophia y Mateo juegan al juego de las monedas: sobre una fila de n monedas de distintos valores, los jugadores alternan turnos y, en cada uno, deben tomar la primera o la última moneda de la fila. El juego termina cuando no quedan monedas, y gana quien acumuló el mayor valor.

Pasaron varios años y Mateo ya tiene 7 años, la misma edad que tenía Sophia cuando empezaron a jugar. Mateo aprendió algoritmos greedy y ahora los aplica: en cada turno elige para sí la más grande de las dos monedas de los extremos. Con esa conducta, el resultado de la partida deja de ser sencillo de controlar, y la estrategia greedy que le servía a Sophia en el trabajo anterior ya no le garantiza el mejor resultado.

Sophia, por su parte, empezó a estudiar programación dinámica y quiere usarla para obtener el máximo valor acumulado posible, sabiendo exactamente cómo va a responder su hermano.

1.2. Objetivo del trabajo

Dada la secuencia de monedas $m_0, m_1, \ldots, m_{n-1}$, sabiendo que Sophia empieza y que Mateo siempre toma la mayor de las dos monedas de algún extremo, se busca determinar qué monedas debe elegir Sophia para obtener el máximo valor acumulado posible. Conviene aclarar desde el principio que el objetivo es maximizar el valor que junta Sophia, y no asegurar que gane. Hay secuencias en las que ninguna estrategia de Sophia le alcanza para ganar: para $[1, 10, 5]$, por ejemplo, el mejor resultado posible de Sophia es 6 contra 10 de Mateo.

1.3. Convenciones y supuestos

A lo largo del informe se usan las siguientes convenciones.

- Dada la secuencia $m_0, m_1, \ldots, m_{n-1}$, cada $m_i > 0$ y todas las monedas tienen diferentes denominaciones.
- Las monedas se numeran desde 0, de modo que la secuencia es $m_0, \ldots, m_{n-1}$. Esto coincide con la indexación de la implementación. El enunciado numera desde 1, pero el cambio es solo de notación.
- Se escribe $[i, j]$ para el tramo contiguo $m_i, m_{i+1}, \ldots, m_j$ donde i y j representan las posiciones (o índices) de los extremos izquierdo y derecho respectivamente . Si $i > j$, el tramo es vacío y no tiene monedas. Su longitud es $j - i + 1$ cuando $i \leq j$ y 0 en caso contrario.
- Se llama $S = \sum_{k=0}^{n-1} m_k$ al valor total de las monedas. Si Sophia junta A, Mateo junta $S - A$, ya que al terminar el juego se tomaron todas las monedas.
- Los valores de las monedas son números arbitrarios. En ningún punto del análisis ni de la demostración se supone que sean distintos entre sí ni que tengan un signo determinado.

2. Análisis del problema y ecuación de recurrencia

2.1. Reglas del juego

Sobre la fila $m_0, \ldots, m_{n-1}$, Sophia y Mateo alternan turnos, y Sophia juega primero. En cada turno, el jugador retira la primera o la última moneda de la fila actual. Sophia puede elegir libremente entre las dos. Mateo, en cambio, aplica siempre la misma regla: compara las dos monedas de los extremos y se queda con la de mayor valor.

2.2. Quién toma decisiones

Dado que Mateo sigue una regla fija, su jugada queda determinada por el estado de la fila. Las únicas decisiones libres de la partida son las de Sophia, que en cada turno elige entre dos opciones.

Importa también qué se pide exactamente. El objetivo es el *máximo valor acumulado* de Sophia, no que gane. Para $[1, 10, 5]$ el valor total es 16. Si Sophia toma el 1, Mateo se queda con el 10 y a ella le toca el 5, con lo que junta 6. Si empieza por el 5, Mateo toma el 10 y a ella le queda el 1, con lo que también junta 6. El máximo posible es 6 contra 10 y Mateo gana en cualquier caso. El algoritmo debe devolver igualmente ese máximo y las monedas que lo producen.

2.3. Por qué una estrategia greedy no alcanza

La estrategia natural es que Sophia tome siempre la mayor de las dos monedas de uno de sus extremos. Esa elección ignora el hecho de que al retirar una moneda, Sophia deja descubierta la moneda que estaba detrás, y es Mateo quien se beneficia de ella.

Consideremos $[6, 20, 1, 5]$, con $S = 32$. Si Sophia aplica la estrategia greedy, la partida es la siguiente.

Turno	Fila al inicio	Juega	Toma
1	$[6, 20, 1, 5]$	Sophia	6
2	$[20, 1, 5]$	Mateo	20
3	$[1, 5]$	Sophia	5
4	$[1]$	Mateo	1

Sophia junta $6 + 5 = 11$ y Mateo $20 + 1 = 21$. Si en cambio Sophia empieza tomando el 5, que es la menor de las dos monedas extremas, la partida cambia por completo.

Turno	Fila al inicio	Juega	Toma
1	$[6, 20, 1, 5]$	Sophia	5
2	$[6, 20, 1]$	Mateo	6
3	$[20, 1]$	Sophia	20
4	$[1]$	Mateo	1

Ahora Sophia junta $5 + 20 = 25$ y Mateo $6 + 1 = 7$. Tomar la moneda más chica en el primer turno le sirvió, porque obligó a Mateo a elegir entre el 6 y el 1 y dejó el 20 al alcance de Sophia. Una decisión local no basta para detectar esto, y por eso hace falta evaluar las consecuencias de cada elección sobre el resto de la partida.

2.4. Estados, subestructura óptima y subproblemas repetidos

Cada jugador retira monedas únicamente de los extremos. Al inicio de cada ronda, Sophia se enfrenta a un tramo contiguo $[i, j]$ y selecciona entre la moneda izquierda (m_i) o la derecha (m_j) la jugada que maximice su ganancia futura acumulada. Luego, Mateo juega sobre el nuevo tramo resultante $[i', j']$ aplicando su estrategia greedy, seleccionando la moneda de mayor valor entre los dos nuevos extremos. De esta manera, cada jugador actúa siguiendo su propio criterio exclusivamente en función del tramo disponible en su turno, sin importar qué monedas fueron retiradas en rondas previas.

El objetivo del problema es encontrar una estrategia de juego para Sophia, en el cual se asegure conseguir la mayor suma acumulada de valores de monedas al finalizar el juego.

2.5. Análisis de una ronda

Al analizar una ronda desde el punto de vista de Sophia, se observa que:

1. Sophia se encuentra con el tramo $[i, j]$ y debe tomar una decisión: retirar la moneda del extremo izquierdo (m_i) o la del extremo derecho (m_j).
2. Después, Mateo observa el tramo resultante y ejecuta su propia regla: compara los dos nuevos extremos y retira el de mayor valor.
3. Tras la jugada de Mateo, se retiran en total 2 monedas de la mesa y luego sigue Sophia sobre un nuevo tramo contiguo más corto $[i', j']$.

Se observa que la mejor jugada futura de Sophia no depende de cuáles monedas fueron retiradas en turnos anteriores ni de que jugadas se hicieron para llegar a ese orden de monedas, sino exclusivamente del tramo de monedas $[i', j']$ que tiene enfrente al momento de volver a jugar. Esto permite definir el siguiente subproblema: para cualquier intervalo $[i, j]$ donde le corresponde mover a Sophia, resolver el juego consiste en determinar la ganancia máxima que ella puede acumular a partir de ese instante, definido como $OPT(i, j)$.

2.6. Paso a paso

2.6.1. Tramo vacío $(i > j)$

No hay monedas para elegir. La ganancia acumulada para Sophia es nula:

$$\text{OPT}(i, j) = 0 \quad \text{si } i > j.$$

2.6.2. Tramo de una moneda $(i = j)$

Solo existe la moneda m_i. Sophia la toma y termina el juego:

$$\text{OPT}(i, i) = m_i.$$

La ganancia de Mateo es nula, dado que no hay más monedas por elegir.

2.6.3. Tramo de dos monedas $(j = i + 1)$

Solo están las monedas m_i y m_{i+1}. Como Sophia busca maximizar su ganancia:

$$\text{OPT}(i, i + 1) = \text{máx}(m_i, m_{i+1}).$$

La moneda restante es para Mateo, por lo que se queda con la moneda de menor valor.

2.6.4. Tramo de tres monedas ($j = i + 2$)

La fila consta de las siguientes monedas: $[m_i, m_{i+1}, m_{i+2}]$.

- **Si Sophia toma la primera moneda (m_i):**
  A Mateo le queda el tramo $[i{+}1, i{+}2]$. Aplicando su regla de greedy, Mateo retira máx(m_{i+1}, m_{i+2}). Dependiendo de su elección, a Sophia le queda una moneda por elegir:

$$\text{Ganancia izquierda} = m_i + \begin{cases} \text{OPT}(i+2, i+2) & \text{si } m_{i+1} > m_{i+2} \quad \text{(Mateo retira } m_{i+1}\text{)}, \\ \text{OPT}(i+1, i+1) & \text{si } m_{i+2} > m_{i+1} \quad \text{(Mateo retira } m_{i+2}\text{)}. \end{cases}$$

  La ganancia resulta $m_i + m_{i+2}$ o $m_i + m_{i+1}$ respectivamente.

- **Si Sophia toma la última moneda (m_{i+2}):**
  A Mateo le queda el tramo $[i, i+1]$. Mateo retira máx(m_i, m_{i+1}), dejando a Sophia:

$$\text{Ganancia derecha} = m_{i+2} + \begin{cases} \text{OPT}(i+1, i+1) & \text{si } m_i > m_{i+1} \quad \text{(Mateo retira } m_i\text{)}, \\ \text{OPT}(i, i) & \text{si } m_{i+1} > m_i \quad \text{(Mateo retira } m_{i+1}\text{)}. \end{cases}$$

  Equivalentemente, $m_{i+2} + m_{i+1}$ o $m_{i+2} + m_i$.

Sophia elige la alternativa que maximice su ganancia acumulada:

$$\text{OPT}(i, i+2) = \text{máx}(\text{Ganancia izquierda}, \text{Ganancia derecha}).$$

2.7. Ecuación de recurrencia

Sea $\text{OPT}(i, j)$ el máximo valor que puede acumular Sophia en el tramo $[i, j]$ cuando juega primero y Mateo aplica su regla. Se busca $\text{OPT}(0, n-1)$.

Respuesta de Mateo. Dado un tramo no vacío $[a, b]$ en el que le toca jugar a Mateo, el tramo que queda después de su jugada es

$$\mu(a, b) = \begin{cases} [a, b-1] & \text{si } m_b \geq m_a \quad \text{(toma la última)}, \\ [a+1, b] & \text{si } m_b < m_a \quad \text{(toma la primera)}. \end{cases}$$

Si el tramo es vacío no hay nada que tomar, y el resultado es nuevamente un tramo vacío.

Resultado de cada elección de Sophia. Si Sophia toma la primera moneda del tramo $[i, j]$, se queda con m_i, queda $[i+1, j]$ y mueve Mateo. Si toma la última, se queda con m_j, queda $[i, j-1]$ y mueve Mateo. Los tramos que le quedan a Sophia en cada caso son

$$\text{P}(i, j) = \mu(i+1, j), \qquad \text{U}(i, j) = \mu(i, j-1).$$

Ecuación. Con la convención de que OPT vale 0 sobre todo tramo vacío, para $i \leq j$ se tiene

$$\text{OPT}(i, j) = \text{máx}\Big(m_i + \text{OPT}\big(\text{P}(i, j)\big),\ m_j + \text{OPT}\big(\text{U}(i, j)\big)\Big).$$

Forma explícita. Al reemplazar μ por su definición se obtiene la forma que se implementa. Para tramos de una y de dos monedas no queda nada por jugar después de Sophia y Mateo, y los valores son directos. Para tramos de tres o más monedas se aplican los casos de μ.

$$\text{OPT}(i, j) = \begin{cases} m_i & \text{si } i = j, \\ \text{máx}(m_i, m_j) & \text{si } j = i+1, \\ \text{máx}\Big(m_i + a(i, j),\ m_j + b(i, j)\Big) & \text{si } j \geq i+2, \end{cases}$$

donde

$$a(i,j) = \begin{cases} \text{OPT}(i+1, j-1) & \text{si } m_j \geq m_{i+1}, \\ \text{OPT}(i+2, j) & \text{si } m_j < m_{i+1}, \end{cases} \qquad b(i,j) = \begin{cases} \text{OPT}(i, j-2) & \text{si } m_{j-1} \geq m_i, \\ \text{OPT}(i+1, j-1) & \text{si } m_{j-1} < m_i. \end{cases}$$

El término $m_i + a(i,j)$ corresponde a que Sophia tome la primera moneda y $m_j + b(i,j)$ a que tome la última. Las condiciones dentro de a y b son la comparación de extremos que hace Mateo sobre el tramo que Sophia le deja. En a, Mateo ve m_{i+1} y m_j. En b, ve m_i y m_{j-1}. Que ambas formas de la recurrencia son equivalentes se comprueba directamente: para $j = i+1$, la forma general con tramos vacíos da $\max(m_i, m_j)$, y para $j \geq i+2$ las dos formas coinciden término a término.

2.8. Orden de resolución y reconstrucción

El cálculo de $\text{OPT}(i,j)$ para un tramo de longitud $\ell \geq 3$ solo necesita valores de tramos de longitud $\ell - 2$ (el tramo que queda tras la jugada de Sophia tiene longitud $\ell - 1$ y tras la de Mateo, $\ell - 2$). Por eso alcanza con completar la tabla por longitudes crecientes: primero los tramos de longitud 1 y 2, que son los casos base, y luego los de longitud $3, 4, \ldots, n$. Cuando se calcula una celda, todas las que necesita ya están disponibles.

Al terminar, el valor $\text{OPT}(0, n-1)$ es la máxima ganancia de Sophia. Para conocer *qué monedas* debe tomar, se recorre la partida desde el tramo $[0, n-1]$: en cada turno de Sophia se vuelve a evaluar cuál de las dos opciones alcanza el máximo, se registra esa elección, se simula la respuesta de Mateo y se continúa con el tramo resultante.

3. Demostración de optimalidad

En esta sección se demuestra que la ecuación de recurrencia de la sección 2 calcula el máximo valor que Sophia puede juntar, y que el algoritmo de reconstrucción devuelve monedas cuya suma es ese máximo. Primero se define con precisión qué significa "el máximo posible", sin recurrir a la recurrencia, y luego se prueba por inducción que ambas cosas coinciden.

3.1. Qué se quiere demostrar

Sea n la cantidad total de monedas en la fila de entrada. Consideremos un tramo $[i, j]$ de $k = j - i + 1$ monedas, en el que Sophia juega primero. Mateo toma decisiones fijas: en cada turno su jugada queda determinada por las dos monedas de los extremos siguiendo su regla greedy. Por lo tanto, lo único que puede variar de una partida a otra son las elecciones de Sophia, que en cada turno toma la primera o la última moneda de la fila. Una partida queda entonces completamente determinada por la sucesión de lados que elige Sophia. Aun si se permitiera que su elección dependiera de lo que ocurrió antes, el resultado sería el mismo, porque lo que ocurrió antes también está determinado por esas elecciones. Es por eso que alcanza con considerar sucesiones fijas de elecciones.

Cada sucesión de elecciones da un total para Sophia, que es la suma de las monedas que toma. Como hay una cantidad finita de sucesiones, existe un máximo. Llamamos $V(i, j)$ al mayor total que puede juntar Sophia sobre las monedas $m_i, \ldots, m_j$, tomado entre todas las sucesiones de elecciones posibles. La definición de V no usa la recurrencia. Lo que se quiere demostrar es que, para todo $0 \leq i \leq j \leq n - 1$,

$$V(i, j) = \mathrm{OPT}(i, j).$$

En particular, $V(0, n - 1) = \mathrm{OPT}(0, n - 1)$ es el máximo que puede juntar Sophia en el problema original.

3.2. Cómo continúa la partida después de una jugada

La demostración se apoya en dos observaciones sobre el juego.

Observación 1: el resto de la partida es el mismo problema. Supongamos que hay $k \geq 3$ monedas, Sophia toma una de las dos del extremo y Mateo responde con su regla. Después de las dos jugadas quedan $k - 2 \geq 1$ monedas, que siguen siendo consecutivas porque ambos tomaron de los extremos, y le vuelve a tocar a Sophia. Las opciones de Sophia y la respuesta de Mateo en lo que resta de la partida dependen únicamente de cuáles son las monedas que quedan, y no de cómo se llegó a esa situación. Por eso, si las monedas que quedan son $m_p, \ldots, m_q$, lo que sucede de ahí en adelante es exactamente el problema original aplicado a $[p, q]$. Además, como Mateo es determinista, el tramo $[p, q]$ que resulta queda fijado por la primera elección de Sophia. La tabla de la sección 2 indica cuál es en cada caso.

Observación 2: el total se separa en dos partes. Si Sophia empieza tomando una moneda de valor c y la partida sigue sobre $[p, q]$, el total de Sophia en esa partida es c más lo que junta sobre $[p, q]$. Además, cualquier sucesión de elecciones válida sobre $[p, q]$ puede usarse como continuación, y toda partida que empieza con esa primera elección se obtiene de este modo. Entonces, el mayor total que puede lograr Sophia empezando por esa moneda es

$$c + V(p, q),$$

porque c es una constante respecto de las elecciones posteriores, y sumar una constante a todos los valores de un conjunto no cambia cuál es el mayor.

3.3. La recurrencia calcula el valor máximo

Optimalidad de la recurrencia: Para todo tramo $[i, j]$ con $0 \leq i \leq j \leq n-1$ se cumple $V(i, j) = \text{OPT}(i, j)$.

Se prueba por inducción fuerte sobre la cantidad de monedas $k = j - i + 1$, con $1 \leq k \leq n$.

Caso base, $k = 1$. Hay una sola moneda y Sophia debe tomarla. El único total posible es m_i, de modo que $V(i, i) = m_i = \text{OPT}(i, i)$.

Caso base, $k = 2$. Hay dos monedas, m_i y m_{i+1}. Sophia toma una de las dos y Mateo se queda con la otra. Los totales posibles para Sophia son m_i y m_{i+1}, y el máximo es $\text{máx}(m_i, m_{i+1})$. Entonces, $V(i, i+1) = \text{máx}(m_i, m_{i+1}) = \text{OPT}(i, i+1)$.

Paso inductivo, $k \geq 3$. Supongamos que $V(p, q) = \text{OPT}(p, q)$ para todo tramo $[p, q]$ de menos de k monedas, y sea $[i, j]$ un tramo de k monedas. Toda partida sobre $[i, j]$ empieza con una de dos elecciones de Sophia, y por lo tanto $V(i, j)$ es el mayor entre los dos mejores totales que se pueden lograr con cada una.

1. *Sophia toma la primera moneda, m_i.* Quedan $m_{i+1}, \ldots, m_j$ y le toca a Mateo, que compara m_{i+1} con m_j. Si $m_j \geq m_{i+1}$, toma m_j y sigue la partida sobre $[i+1, j-1]$. Si $m_j < m_{i+1}$, toma m_{i+1} y sigue sobre $[i+2, j]$. En ambos casos el tramo resultante tiene $k-2$ monedas, que son menos que k. Por la observación 2, el mejor total empezando por m_i es $m_i + V(\cdot)$ sobre ese tramo, y por la hipótesis inductiva esto es $m_i + a(i, j)$, donde $a(i, j)$ es exactamente el valor OPT del tramo que corresponde en cada caso.

2. *Sophia toma la última moneda, m_j.* Quedan $m_i, \ldots, m_{j-1}$ y Mateo compara m_i con m_{j-1}. Si $m_{j-1} \geq m_i$, toma m_{j-1} y sigue la partida sobre $[i, j-2]$. Si $m_{j-1} < m_i$, toma m_i y sigue sobre $[i+1, j-1]$. De nuevo, el tramo resultante tiene $k-2$ monedas, y por la observación 2 y la hipótesis inductiva el mejor total empezando por m_j es $m_j + b(i, j)$.

Por lo tanto,

$$V(i, j) = \text{máx}\big(m_i + a(i, j),\ m_j + b(i, j)\big) = \text{OPT}(i, j),$$

que es el tercer caso de la ecuación de recurrencia. Con esto se completa la inducción.

La demostración se apoya en tres hechos: Mateo es determinista, lo que permite saber con exactitud qué tramo queda después de cada elección de Sophia; el resto de la partida depende solo de ese tramo; y el mejor total que Sophia puede lograr desde un tramo ya es, por hipótesis inductiva, el valor OPT de ese tramo. Los valores concretos de las monedas intervienen únicamente a través de las comparaciones de Mateo, que se resuelven exhaustivamente en las cuatro situaciones de la tabla.

3.4. El algoritmo calcula la recurrencia

Corrección del llenado de la tabla: Al finalizar el llenado, la celda `matriz_solucion[j][i]` contiene $\text{OPT}(i, j)$ para todo $0 \leq i \leq j \leq n-1$.

Se razona por inducción sobre la cantidad de monedas $k = j - i + 1$. Los tramos de una y de dos monedas se cargan directamente con los valores de los casos base de la ecuación. Sea $k \geq 3$ y supongamos que las celdas de todos los tramos de menos de k monedas ya contienen su valor correcto.

El bucle externo procesa las diferencias $j - i = 2, 3, \ldots, n-1$ en orden creciente, es decir, los tramos de $k = 3, 4, \ldots, n$ monedas en ese orden. Cuando llega al tramo $[i, j]$, ya se calcularon todos los de menos monedas. Entre ellos están los tres que puede necesitar la fórmula, $[i+1, j-1]$, $[i+2, j]$ e $[i, j-2]$, que tienen todos $k-2$ monedas. La expresión `opt_1` del código es $m_j + b(i, j)$ y `opt_2` es $m_i + a(i, j)$, con las mismas comparaciones de la tabla, y la celda se completa con el mayor de los dos. Por la hipótesis inductiva, el valor almacenado es $\text{OPT}(i, j)$.

3.5. La reconstrucción alcanza el máximo

La función `reconstruir_solucion` parte del tramo $[0, n-1]$. Si el tramo tiene una moneda, la toma. Si tiene dos, toma la de mayor valor. Si tiene tres o más, calcula $m_j + b(i,j)$ y $m_i + a(i,j)$ con los valores de la tabla, toma la última moneda cuando el primero es estrictamente mayor y la primera en caso contrario, y continúa con el tramo que resulta de aplicar la respuesta de Mateo, que es el mismo que figura en la tabla de la sección 2.

Corrección de la reconstrucción: Las monedas que devuelve `reconstruir_solucion` sobre $[0, n-1]$ suman exactamente $\text{OPT}(0, n-1)$. En consecuencia, son un conjunto de monedas con el que Sophia alcanza el máximo valor acumulado posible.

Se prueba por inducción sobre la cantidad de monedas k que la función, aplicada a un tramo $[i,j]$, devuelve monedas que suman $\text{OPT}(i,j)$.

Si $k = 1$, devuelve $m_i = \text{OPT}(i,i)$. Si $k = 2$, devuelve $\text{máx}(m_i, m_{i+1}) = \text{OPT}(i, i+1)$.

Sea $k \geq 3$ y supongamos el resultado para menos monedas. La función compara $m_j + b(i,j)$ con $m_i + a(i,j)$ y elige la moneda x que da el mayor. Por la ecuación de recurrencia, ese mayor valor es $\text{OPT}(i,j)$. Luego continúa sobre el tramo $[p,q]$ que queda después de la respuesta de Mateo, de $k-2 \geq 1$ monedas. Por hipótesis inductiva, las monedas que devuelve para $[p,q]$ suman $\text{OPT}(p,q)$, y como el valor elegido era $x + \text{OPT}(p,q)$, el total devuelto es $x + \text{OPT}(p,q) = \text{OPT}(i,j)$.

4. Algoritmo propuesto y complejidad

4.1. Estructura de datos

El algoritmo guarda los valores de OPT en una matriz cuadrada de $n \times n$ posiciones. En la implementación, la celda `matriz_solucion[fin][inicio]` contiene el valor OPT(`inicio`, `fin`). Solo se utilizan las celdas con `fin` $\geq$ `inicio`, es decir, la parte triangular inferior de la matriz, ya que los tramos con `inicio` $>$ `fin` son vacíos y no se almacenan. En el resto de esta sección se escribe OPT(i, j) para referirse a la celda `matriz_solucion[j][i]`, con $i =$ `inicio` y $j =$ `fin`.

4.2. Descripción del algoritmo

El algoritmo tiene dos etapas.

1. **Llenado de la tabla.** Se inicializa la diagonal con los tramos de una moneda, OPT$(i, i) = m_i$, y la subdiagonal con los de dos monedas, OPT$(i, i+1) = \max(m_i, m_{i+1})$. Luego se recorren los tramos por longitud creciente, desde 3 hasta n, y para cada uno se aplica la ecuación de recurrencia. Cada celda se calcula con dos sumas, dos comparaciones para modelar la respuesta de Mateo y un máximo.
2. **Reconstrucción.** Partiendo del tramo $[0, n-1]$, en cada turno de Sophia se recalculan los dos valores que compiten en la recurrencia y se toma la moneda que da el mayor. Se registra la moneda y el extremo del que se tomó, se simula la jugada de Mateo con su regla y se continúa con el tramo que queda. El proceso termina cuando el tramo es de una o dos monedas, que se resuelven directamente.

4.3. Código

El código que sigue corresponde a las dos funciones centrales. Se omiten la lectura del archivo de entrada y la impresión del desarrollo de la partida, que no forman parte del algoritmo. En `maxima_ganancia_sofia`, la variable `j` cumple el rol de i (inicio del tramo), `k` el de j (fin del tramo), y `i` es la diferencia entre ambos extremos, que crece de a uno y fija la longitud del tramo que se calcula. La variable `opt_1` es el valor de que Sophia tome la última moneda, y `opt_2` el de que tome la primera.

```
def maxima_ganancia_sofia(monedas, mostar_detalle=False):
    n = len(monedas)
    matriz_solucion = [[None] * n for _ in range(n)]

    for i in range(n):
        matriz_solucion[i][i] = monedas[i]

    for i in range(n - 1):
        matriz_solucion[i+1][i] = max(monedas[i], monedas[i+1])

    for i in range(2, n):
        k = i
        for j in range(n - i):
            opt_1 = monedas[k] + (matriz_solucion[k-2][j] if monedas[k-1] >=
    monedas[j] else matriz_solucion[k-1][j+1])
            opt_2 = monedas[j] + (matriz_solucion[k-1][j+1] if monedas[k] >=
    monedas[j+1] else matriz_solucion[k][j+2])
            matriz_solucion[k][j] = max(opt_1, opt_2)
            k += 1

    solucion = []
    lados = []
    reconstruir_solucion(monedas, matriz_solucion, 0, n-1, solucion, lados)
    if mostrar_detalle:
        mostrar_pasos(lados, monedas)
    return solucion
```

```python


def reconstruir_solucion(monedas, matriz, inicio, fin, solucion, lados):
    if fin == inicio:
        solucion.append(monedas[inicio])
        lados.append("inicio")
        return solucion

    if fin - inicio == 1:
        if monedas[fin] > monedas[inicio]:
            solucion.append(monedas[fin])
            lados.append("fin")
            return solucion

        solucion.append(monedas[inicio])
        lados.append("inicio")
        return solucion

    opt_1 = monedas[fin] + (matriz[fin-2][inicio] if monedas[fin-1] >= monedas[inicio] else matriz[fin-1][inicio+1])
    opt_2 = monedas[inicio] + (matriz[fin-1][inicio+1] if monedas[fin] >= monedas[inicio+1] else matriz[fin][inicio+2])

    if opt_1 > opt_2:
        solucion.append(monedas[fin])
        lados.append("fin")
        if monedas[fin-1] >= monedas[inicio]:
            return reconstruir_solucion(monedas, matriz, inicio, fin-2, solucion, lados)
        return reconstruir_solucion(monedas, matriz, inicio+1, fin-1, solucion, lados)

    solucion.append(monedas[inicio])
    lados.append("inicio")
    if monedas[fin] >= monedas[inicio+1]:
        return reconstruir_solucion(monedas, matriz, inicio+1, fin-1, solucion, lados)
    return reconstruir_solucion(monedas, matriz, inicio+2, fin, solucion, lados)
```

La lista `solucion` acumula, en orden, las monedas que toma Sophia, y `lados` registra si cada una se tomó del inicio o del fin del tramo vigente. La segunda lista no interviene en el cálculo y solo sirve para que la salida del programa pueda mostrar cada turno sin ambigüedad cuando las dos puntas tienen el mismo valor. Como el tramo siguiente se obtiene con las mismas comparaciones que usa la recurrencia, la partida que se reconstruye es exactamente la que el modelo describe.

La relación entre el código y la ecuación es directa. La expresión `opt_1` equivale a $m_j + b(i, j)$ y `opt_2` a $m_i + a(i, j)$, con $j =$ `fin` e $i =$ `inicio`. Al finalizar el llenado, la celda `matriz_solucion[n-1][0]` contiene $\mathrm{OPT}(0, n-1)$. Cuando `opt_1` y `opt_2` son iguales, el código toma la primera moneda; el valor total no depende de esa elección.

4.4. Ejemplo de ejecución

Sea la secuencia $[8, 5, 20, 9]$, con $S = 42$. Los casos base son $\mathrm{OPT}(0, 0) = 8$, $\mathrm{OPT}(1, 1) = 5$, $\mathrm{OPT}(2, 2) = 20$, $\mathrm{OPT}(3, 3) = 9$, y para los tramos de dos monedas $\mathrm{OPT}(0, 1) = 8$, $\mathrm{OPT}(1, 2) = 20$, $\mathrm{OPT}(2, 3) = 20$. Para los tramos de tres monedas:

- $\mathrm{OPT}(0, 2)$: tomar la primera da $8 + a(0, 2)$. Como $m_2 = 20 \geq m_1 = 5$, Mateo toma el 20 y queda $[1, 1]$, por lo que $a(0, 2) = \mathrm{OPT}(1, 1) = 5$ y el total es 13. Tomar la última da $20 + b(0, 2)$. Como $m_1 = 5 < m_0 = 8$, Mateo toma el 8 y queda $[1, 1]$, de modo que $b(0, 2) = 5$ y el total es 25. Entonces $\mathrm{OPT}(0, 2) = 25$.
- $\mathrm{OPT}(1, 3)$: tomar la primera da $5 + a(1, 3)$. Como $m_3 = 9 < m_2 = 20$, Mateo toma el 20 y queda $[3, 3]$, de modo que el total es $5 + 9 = 14$. Tomar la última da $9 + b(1, 3)$. Como

$m_2 = 20 \geq m_1 = 5$, Mateo toma el 20 y queda $[1, 1]$, de modo que el total es $9 + 5 = 14$. Entonces $\text{OPT}(1, 3) = 14$.

Para el tramo completo, tomar la primera moneda da $8 + a(0, 3)$. Como $m_3 = 9 \geq m_1 = 5$, Mateo toma el 9 y queda $[1, 2]$, de modo que el total es $8 + \text{OPT}(1, 2) = 28$. Tomar la última da $9 + b(0, 3)$. Como $m_2 = 20 \geq m_0 = 8$, Mateo toma el 20 y queda $[0, 1]$, de modo que el total es $9 + \text{OPT}(0, 1) = 17$. Por lo tanto, $\text{OPT}(0, 3) = 28$. La tabla completa, con i en las filas y j en las columnas, es la siguiente.

$i \backslash j$	0	1	2	3
0	8	8	25	**28**
1		5	20	14
2			20	20
3				9

La reconstrucción recorre la partida de la siguiente manera.

Turno	Tramo	Sophia toma	Mateo toma
1	$[8, 5, 20, 9]$	8 (da 28 frente a 17)	9
2	$[5, 20]$	20	5

Sophia junta $8 + 20 = 28$ y Mateo $9 + 5 = 14$, de modo que, en este caso, Sophia gana. Una estrategia greedy habría tomado el 9 en el primer turno, ya que $9 > 8$. Con ello, Mateo habría tomado el 20, y Sophia terminaría con $9 + 8 = 17$.

4.5. Complejidad temporal

Se mide el tiempo en cantidad de operaciones elementales sobre las monedas, suponiendo que sumar y comparar valores cuesta tiempo constante. Se analiza cada parte del algoritmo por separado.

Etapa	Cantidad de pasos	Costo
Creación de la matriz $n \times n$	n^2 celdas	$O(n^2)$
Tramos de una moneda	n celdas	$O(n)$
Tramos de dos monedas	$n - 1$ celdas	$O(n)$
Tramos de tres o más monedas	$\frac{(n-1)(n-2)}{2}$ celdas	$O(n^2)$
Reconstrucción	$\lceil n/2 \rceil$ llamadas	$O(n)$

Llenado de la tabla. El bucle externo recorre las diferencias $d = \texttt{fin} - \texttt{inicio}$ desde 2 hasta $n - 1$. Para cada d, el bucle interno calcula los $n - d$ tramos de esa longitud, y cada cálculo tiene costo constante porque consiste en un número fijo de sumas, comparaciones y accesos a la matriz. La cantidad de celdas calculadas en este bucle es

$$\sum_{d=2}^{n-1} (n - d) = (n - 2) + (n - 3) + \cdots + 1 = \frac{(n - 1)(n - 2)}{2}.$$

Junto con las n celdas de longitud 1 y las $n - 1$ de longitud 2, se completan en total

$$n + (n - 1) + \frac{(n - 1)(n - 2)}{2} = \frac{n(n + 1)}{2}$$

celdas, es decir, todos los tramos posibles. Cada una se calcula una única vez y en tiempo constante, de modo que el llenado cuesta $\Theta(n^2)$.

Reconstrucción. Cada llamada de `reconstruir_solucion` resuelve un turno de Sophia y la respuesta de Mateo, de modo que el tramo se acorta en dos monedas, y realiza una cantidad constante de operaciones (el `append` amortizado también es constante). Las llamadas terminan cuando el tramo es de una o dos monedas, por lo que son $\lceil n/2 \rceil$ en total. El costo es $O(n)$.

Total. El tiempo del algoritmo es $\Theta(n^2) + O(n) = \Theta(n^2)$. Como el algoritmo completa siempre todas las celdas y las comparaciones solo determinan qué rama de la expresión condicional se evalúa, el costo no depende de cómo estén distribuidos los valores de las monedas. Solo depende de n. No hay entonces mejor ni peor caso: el tiempo es cuadrático para cualquier entrada de tamaño n.

Comparación con la alternativa exhaustiva. Probar todas las sucesiones de elecciones de Sophia requiere recorrer hasta $2^{\lceil n/2 \rceil}$ partidas, y cada una cuesta $O(n)$ para simularse. La programación dinámica reduce ese costo a $O(n^2)$ porque evalúa cada tramo una sola vez.

4.6. Complejidad espacial

La matriz formalmente ocupa n^2 posiciones, por lo que el espacio asintótico es $O(n^2)$. Pero aún reservando de esa forma la matriz en memoria, el algoritmo solo completa y consulta la porción de la triangular inferior que corresponde a los subproblemas válidos (donde $inicio \leq fin$), usando exactamente las $\frac{n(n+1)}{2}$ celdas.

Las listas `solucion` y `lados` tienen a lo sumo $\lceil n/2 \rceil$ elementos cada una, y la pila de llamadas recursivas de la reconstrucción tiene una profundidad de $\lceil n/2 \rceil$. Estos términos son $O(n)$ y quedan absorbidos por el de la matriz, de modo que el espacio total es $O(n^2)$.

Esta cota se podría reducir, ya que cada celda depende solo de las de longitud inferior en dos unidades. Sin embargo, la reconstrucción de las monedas necesita acceder a toda la tabla, y se prefirió conservarla completa por simplicidad.

5. Análisis Empírico y Mediciones

Para corroborar la complejidad teórica analizada en la sección 4, se realizaron mediciones del tiempo de ejecución usando el set de datos provistos por la cátedra. Los archivos de prueba proporcionados contienen secuencias de monedas con longitudes que varían desde $n = 5$ hasta $n = 10000$.

Las pruebas se ejecutaron aislando el algoritmo principal de la solución provista al problema, garantizando que los tiempos reflejen únicamente el procesamiento lógico. Los tiempos obtenidos se graficaron en función del a cantidad de monedas n.

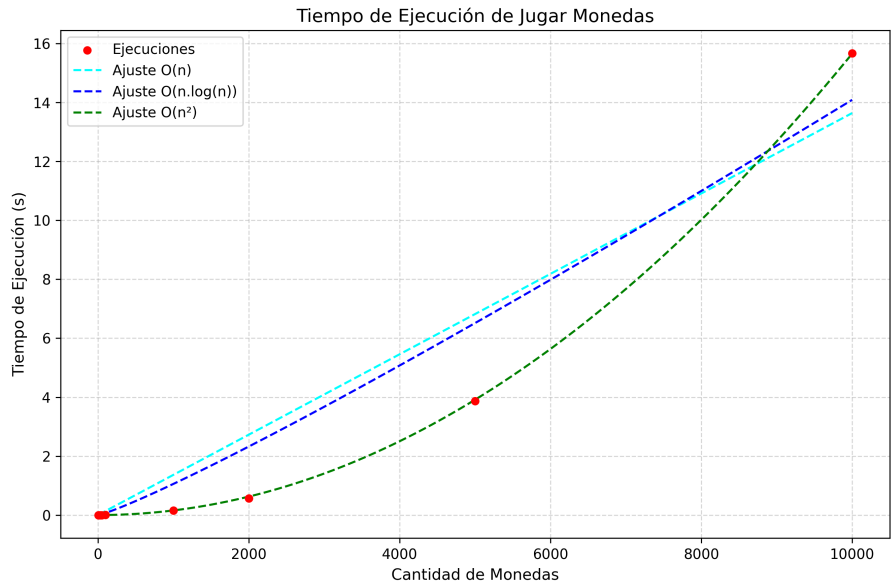


Figura 1: Tiempo de ejecución en función de la cantidad de monedas (n)

Como se puede observar en el gráfico resultante, el crecimiento del tiempo termina formando una parábola a medida que aumenta el tamaño de la entrada, por lo que más allá de las decisiones que toma el algoritmo para tomar un valor u otro, lo condicionante del tiempo termina siendo el valor de entrada n. Este comportamiento coincide perfectamente con la complejidad $\Theta(n^2)$ deducida matemáticamente, demostrando así, que el comportamiento en la práctica del doble bucle anidado para completar la matriz termina siendo el factor dominante en el rendimiento de la solución.

6. Conclusiones

Durante el desarrollo del trabajo práctico, se evidencia de manera clara cómo el enfoque de optimización exacta usada en esta entrega es superior a la heurística del trabajo anterior. La evolución del juego de las monedas demostró empíricamente que el algoritmo Greedy, usado por Mateo, resulta ineficiente para grantizar el óptimo global cuando las decisiones tempranas habilitan jugadas desproporcionadamente ventajosas para el oponente en el futuro. Al basarse únicamente en decisiones locales y de corto plazo, la estrategia de Mateo termina permitiéndole a la de Sofia aprovechar de la natrualeza determinista de sus respuesta.

Para resolverlo, la implementación con programación dinámica resultó ser la herramienta ideal. Al modelar el problema mediante una ecuación de recurrencia, logramos identificar la subestructura óptima del juego; el hecho de que la mejor jugada en un tramo de monedas depende invariablemente de las soluciones óptimas de los tramos más pequeños. Al calcular estas soluciones desde los casos base hacia arriba mediante un enfoque ascendente, evitamos el recálculo masivo de subproblemas superpuestos, reduciendo drásticamente la carga computacional.

Desde la perspectiva del análisis de eficiencia, el diseño algorítmico cumplió con lo esperado matemáticamente. La complejidad temporal del rellenado de la matriz demostró ser estrictamente $\Theta(n^2)$ algo que fue sólidamente respaldado por las mediciones empíricas realizadas, donde el tiempo de ejecución dibujó una parábola perfecta en función del tamaño de la entrada n. Esto corrobora que, sin importar los valores particulares de las monedas o si nos encontramos en el peor caso, el comportamiento asintótico está dominado por la cantidad de inputs a procesar. Por otro lado, la complejidad espacial se mantuvo acotada a $O(n^2)$ gracias al almacenamiento de los estados en una matriz bidimensional.

Además, nuestro análisis exhaustivo demostró un aspecto fundamental; jugar de forma óptima maximiza la ganancia, pero no altera las limitaciones instrínsecas del tablero inicial. Como se evidenció en casos particulares como [1, 10, 5], el resultado de ciertas distribuciones de valores hace imposible la victoria para el primer jugador, independientemente de lo perfecta que sea su estrategia. Nuestro algoritmo cumple su objetivo principal, el no forzar victorias imposibles sino asegurar siempre el máximo valor acumulado matemáticamente posible bajo las reglas dadas. En síntesis, este trabajo no solo consolida la aplicación práctica de recurrencias y reconstrucción de soluciones óptimas de subproblemas, sino que reafirma la importancia de las demostraciones formales por inducción para garantizar la correctitud del software construido. La transición de una lógica heurística inestable hacia un modeflo estructurado y predecible ejemplifica el núcleo de diseño eficiente de algoritmos.